

WoodFlow Handbook

Product Vision · Design System · Architecture · Development Process

Version	1.0 — Frozen (Approved)
Status	Official project reference, approved by the Product Owner
Owner	Piotr
Last Updated	2026-08-12
Single source of truth for	the entire WoodFlow project

This is the single source of truth for the WoodFlow project — product vision, design system, module documentation, architecture, and development process. It supersedes every standalone document it was assembled from; those are preserved at `docs/archive/` for their original reasoning and evidence, not as an alternate source of truth. **If this handbook and an archived document ever disagree, this handbook wins.**

What the v1.0 freeze certifies, precisely: the chapter structure, the Product Principles (Chapter 3), and every chapter marked  below are locked as approved and current. Freezing this document does **not** retroactively approve the content of any chapter still marked , , or  — those remain exactly as open as their own marker says, inside a now-frozen shell, and each still needs its own Phase 1 → Phase 2 approval (Chapter 23) before implementation, unchanged by the freeze itself.

How to read the status markers

Marker	Meaning
 Approved & Implemented	Live in the app today, verified against the code
 Direction approved, not implemented	Concept approved; no Phase 2 build started or approved
 Draft — awaiting review	Proposed, not yet discussed with the Project Owner in this form
 Future Phase — Not Approved	An idea, explicitly not scheduled, not to be built without a separate decision

Nothing marked , , or  gets implemented without explicit approval — see Chapter 23.

Table of Contents

1. Product Vision	
2. Stage 1 Philosophy — Evolution, Not Revolution	
3. Product Principles	
4. UI / UX Design System	
5. Navigation — Command Hub	
6. Visual Identity	
7. Themes — Light & Dark	
8. Responsive Design — Phone / Tablet / Desktop	 / 
9. Warehouse Module	
10. Inventory Management	
11. QR System	

12. Shopping List	✓
13. Calculators	✓
14. Export	✓
15. Dashboard	✓
16. Entity Editing	
17. Animations & Motion	
18. Accessibility	✓
19. Performance Rules	✓
20. Flutter Architecture	✓
21. Coding Standards	✓
22. Folder Structure	✓
23. Development Workflow	✓
24. Future Ideas	
25. Decision Log	
26. Changelog	

1. Product Vision

WoodFlow is not "another inventory app." It is meant to become the most beautiful warehouse management application for woodworking and furniture manufacturers — recognizable on sight, the way Linear, Notion, or Stripe are recognizable in their categories.

What it should communicate immediately: precision, craftsmanship, professionalism, modern technology, industrial elegance.

Reference points — read as restraint, not decoration: Apple, Linear, Notion, Tesla, Autodesk, professional CAD software. None of these earn their identity through busy decoration — they earn it through consistent restraint, precise typography, and calm, deliberate motion. Every visual decision in this handbook is judged against that standard.

Long-term goal: an application a workshop employee finds simple, a company owner finds beautiful, and a customer immediately trusts — recognized eventually not only for functionality but for its own industrial visual identity.

Current roadmap: Stage 1 / FREE — 15 steps ("Kroki"), 14 shipped, 1 remaining (Cut Optimizer, Krok 15). Full canonical status: Chapter 26.

2. Stage 1 Philosophy — Evolution, Not Revolution

Every major decision this project has made has **extended an existing pattern rather than replacing it**, evidenced directly, not just stated:

- The Design System (Ch. 4) restyled screens that were explicitly built "prove it works, don't polish it yet" — it didn't rebuild them.
- `get_it` was kept as the sole DI mechanism over switching to Riverpod (Ch. 20) specifically to avoid a project-wide rewrite for a marginal architectural gain.
- The proposed Command Hub (Ch. 5) reuses Flutter's own `NavigationRail` for landscape/tablet rather than a bespoke navigation widget.
- Migrations are append-only, one new versioned file per change (Ch. 20) — schema history is never rewritten, only added to.
- The Ledger is append-only by the same logic, applied to data instead of code: a correction is a new entry, never an edit (Ch. 10, Ch. 20).

Practical rule this implies: when proposing anything new, first ask whether an existing pattern already solves the shape of the problem. If yes, extend it. Introduce a genuinely new pattern only when nothing existing fits — and say so explicitly when that happens, rather than silently declaring extra machinery necessary.

The same rule applies to this handbook itself — this is Principle 10 (single source of truth, Chapter 3) put into practice, not a separate rule: before introducing a new chapter, section, principle, document, or architectural pattern, first determine whether the handbook already solves the same problem. If it does, extend the existing content rather than creating a duplicate. A second document (or a second section) covering the same ground is exactly how a "single source of truth" quietly stops being one — every prior consolidation in this handbook's own history (Chapter 26's Archive note, the four superseded design documents) exists because that already happened once and had to be corrected. Practical test before adding anything new to this document: "does an existing chapter or section already own this topic?" If yes, extend it there; if the right home is genuinely unclear, that ambiguity is itself worth resolving explicitly rather than picking a location and hoping.

3. Product Principles

Stated directly by the Project Owner. These 12 are **immutable** — every future feature, screen, and design decision in WoodFlow is measured against them, not the other way around. When a proposal conflicts with one of these, the principle wins; the proposal gets revised, not the principle.

1. **Evolution, not revolution.** Extend an existing, proven pattern before inventing a new one. See Chapter 2 for the evidence this is already how the project works, not just an aspiration.
2. **Simplicity over complexity.** The simpler solution is the correct default; complexity has to earn its place by solving a real problem simplicity can't.
3. **Industrial-first design.** Every visual and interaction decision is judged against a workshop/warehouse context first — not against what looks good in a design tool on a desk. See Chapter 4.6's User Experience Principles.
4. **Fast, intuitive workflow.** A returning user should never have to re-learn where something is. Speed and predictability outrank novelty, every time — this is why Chapter 17 rejects several animation ideas outright.
5. **Every feature must solve a real woodworking problem.** Not "would be nice," not "competitors have it" — a named, real problem for a wood/furniture manufacturer. This is the first filter any Chapter 24 idea has to pass before it can move toward approval.
6. **Mobile-first, responsive everywhere.** Every screen is designed for a phone in a warehouse first, then adapted outward to landscape/tablet/desktop (Chapter 8) — never designed for a large screen and shrunk down.
7. **Performance before visual effects.** If a visual idea and the 60 FPS target (Chapter 19) conflict, the visual idea loses, full stop — not "loses unless it's really nice."
8. **No implementation without explicit approval.** Formalized as the Phase 1 / Phase 2 process in Chapter 23. This applies to the Project Owner's own new ideas too, not just Claude Code's proposals — a "final decision" on direction is not the same as a go-ahead to build.
9. **Future ideas must never affect current production.** Everything in Chapter 24 is explicitly, visibly separated (🌀) from anything ✅ or 🟡 — a future idea existing in this document must never be mistaken for a scheduled one, and must never gate or complicate work that's actually shipping now.
10. **This handbook is the single source of truth for the WoodFlow project.** If any other document, comment, or prior conversation disagrees with this handbook, this handbook wins (see the header above) — and if this handbook is ever unclear or silent on something, that's treated as a gap to fix here, not license to improvise elsewhere. The operational rule this implies for *extending* the handbook itself is elaborated in Chapter 2, not repeated here as a second, separate rule.
11. **The operator should never have to guess the next action.** The interface should make the correct action obvious. Ambiguity about "what do I do here" is treated as a defect, the same weight as a functional bug.
12. **Every interaction must have a measurable purpose.** If a click, dialog, screen, or workflow does not create enough value, it should be simplified or removed. "It's already there" is not, on its own, a reason to keep something.

4. UI / UX Design System

Status: ✔ **Approved & Implemented.** Every screen in `lib/presentation/` uses the `WF*` component library (`lib/presentation/design_system/`).

4.1 Spacing (`WFSpacing`)

Token	Value	Role
<code>xs</code>	4dp	icon-to-text gap
<code>sm</code>	8dp	closely related elements
<code>md</code>	12dp	list-tile padding, form-field gaps
<code>lg</code>	16dp	screen edge padding — default
<code>x1</code>	24dp	gap between sections
<code>xx1</code>	32dp	generous empty/onboarding screens only

4.2 Typography

`Theme.of(context).textTheme.*` roles only, never an inline `TextStyle`. Titles → `titleLarge`. List-tile primary text → `titleMedium` / `bodyLarge`. Subtitles → `bodyMedium`. Section headers → `labelMedium`. Counts/timestamps → `labelSmall`. Muted text → `colorScheme.onSurfaceVariant`.

4.3 Color

Seed: `0xFF1F4D3D` (Forest Green). Every functional color is a semantic `colorScheme` role:

Role	Source
Destructive / error	<code>colorScheme.error</code>
Warning (low stock, stale materials)	<code>colorScheme.tertiary</code>
Success / good fill level	<code>colorScheme.primary</code>
Secondary/muted text	<code>colorScheme.onSurfaceVariant</code>

This is what makes dark mode (Ch. 7) possible without touching every screen.

4.4 Elevation, radius, icons

Elevation: Material 3 defaults only (cards 1dp, dialogs/sheets 3dp, FAB 3dp, app bar 0dp flat). Radius (`WFRadius`): `small` 8dp (chips), `medium` 12dp (cards/dialogs/lists), `large` 16dp top-only (bottom sheets). Icons: outlined Material family, one concept → one icon, never reused across entity types. Sizes: 24dp standard, 20dp dense/nested, 96dp QR display, 18–20dp inline buttons.

4.5 Components (`WF*` library)

`WFButton` (4-role: primary `FilledButton`, secondary `OutlinedButton`, tertiary `TextButton`, destructive red `TextButton` — `ElevatedButton` retired), `WFCard`, `WFDialog`, `WFConfirmationDialog`, `WFTextField`,

`WFTopBar`, `WFFloatingActionButton`, `WFListTile`, `WFSectionHeader`, `WFStatusChip`,
`WFProgressIndicator`, `WFSearchBar`, `WFBottomSheet`, `WFSnackbar`, `WFEmptyState`, `WFLoadingState` —
one barrel import (`design_system.dart`).

- Confirmations: `showWFConfirmationDialog` — title = question, content = consequence, Cancel + red destructive verb, never generic "OK."
- Create/edit forms: `showWFBottomSheet` — replaced `AlertDialog` forms for more keyboard room and thumb-reach.
- Lists: `WFListTile`, standard density top-level, `dense: true` nested (Boards/Offcuts inside a Slot).

4.6 User Experience Principles

Eight principles every screen is checked against — not aspirational copy, a working checklist. These are Chapter 3's Product Principles (specifically #3, #4, #6, #7, and now #11) applied at screen-design granularity, not a second, independent principle set — where one of the eight below and a Chapter 3 principle appear to overlap, they're the same rule at two levels of resolution, not two rules to reconcile. Several also extend the three original findings from the Design System audit (still true, folded in below rather than kept as a separate list).

1. **Mobile-first.** Every screen is designed for a phone first, full stop — landscape/tablet/desktop (Chapter 8) are adaptations of the mobile design, never the other way around.
2. **Workshop-first.** The design target is a person on a warehouse floor, not a person at a desk. The Warehouse → Rack → Slot → Board/Offcut hierarchy and its icon "zoom in" matches how an operator actually thinks about finding material — every navigation decision reinforces this, never fights it.
3. **Fast interaction.** A worker scanning and shelving material all shift benefits from speed and predictability more than decoration — this is what governs Chapter 17's animation rejections, not a preference against motion in general.
4. **Minimal taps.** Every additional tap on a frequent action is a real cost, multiplied by every repetition across a shift. This is the concrete reasoning behind keeping QR Scanner outside the Command Hub (Chapter 5.6) — it's too frequent to bury one tap deeper.
5. **One-hand usage.** Primary actions and the FAB stay reachable in the lower two-thirds of the screen — the zone a thumb can reach while the other hand is holding material. The one still-open gap this principle flags: anything living at the very top of a tall phone screen (an old AppBar icon row was the original example — a fix is proposed, not yet approved, in Chapter 4.7/Chapter 5).
6. **Glove-friendly operation.** Touch targets never shrink below Material's 48×48dp default for density reasons (Chapter 18) — a workshop is not always a bare-hands environment, and a "compact" layout that assumes precise bare-finger taps is assuming the wrong thing about who's using it.
7. **Readability from distance.** Text and status signals need to read correctly at arm's length in inconsistent workshop lighting, not just up close in ideal conditions — the reasoning behind status color never being the *only* signal (Chapter 18) and behind keeping decorative motif opacity strictly in the 2–5% band (Chapter 6) so it never competes with real content for attention.
8. **Consistency over visual effects.** A novel transition or a clever one-off interaction loses to a boring, consistent, already-learned pattern every time. This is why Chapter 17 has a hard animation budget and an explicit rejected list, not just an approved list — consistency is treated as a design value in its own right, not the absence of one.

4.7 The 7-icon AppBar question — proposed resolution pending approval

The original Design System document left `WarehouseListScreen`'s 7-action `AppBar` as an open question.

Chapter 5's Command Hub proposes to resolve it — replacing that row entirely — but Chapter 5 is still  draft. The question stays genuinely open until Chapter 5 is approved; this section records the proposed direction, not a finished decision.

5. Navigation — Command Hub

Status: 📄 **Draft** — **awaiting review**, per Chapter 23's process. This chapter describes **behavior** — what a user sees and does — not Flutter implementation details; where an implementation shape is noted (widget names, animation controllers), it's a secondary note for whoever builds Phase 2, not the point of the section.

5.1 The decision

The current `AppBar` icon row is replaced by one branded navigation system. Navigation itself becomes a recognizable WoodFlow brand element, not just a functional necessity.

5.2 Portrait — Command Hub

Behavior: a single floating WoodFlow button, always present, opens the Command Hub. Pressing it dims the current screen, the button itself expands into a panel, and a grid of module tiles appears — the interaction reads as one continuous motion, the panel visibly *coming from* the button, not a generic menu appearing from nowhere. QR Scanner is never inside this panel — see §5.6.

- Floating WoodFlow Button — exact placement (bottom-center vs. bottom-trailing) open, §5.8.
- Sequence: scrim dims the screen → button scales/morphs into a panel anchored at its own origin → the module grid fades/slides in with a ~20–30ms stagger between tiles.
- Module grid, in order: **Dashboard, Warehouses, Shopping List, Calculators, Export, Settings.**
- Closing mirrors opening in reverse — the panel collapses back into the button, not a generic dismiss/fade-out, so the motion always reads as "the button," never as "a screen that happened to close."
- Implementation note (Phase 2, not the point of this section): `OverlayEntry`, not a new route — chrome-level state, doesn't disturb the underlying `Navigator` stack. One `AnimationController`, ~280–320ms total (the one deliberate exception to Ch. 17's ≤250ms budget — a brand moment, not a routine repeated action). Respects reduce-motion.

5.3 Landscape

Behavior: the Command Hub is replaced entirely — there is no floating menu in landscape. A permanent navigation rail occupies the left edge (icons + labels, all six modules always visible, no tap needed to see or reach any of them), and the actual screen content ("the workspace") fills the remaining width to the right. Rotating the phone doesn't just stretch the portrait layout into a wider frame — it switches to a structurally different, rail-based navigation model built for the extra width.

Implementation note: Flutter's built-in `NavigationRail`. A persistent rail costs zero taps versus the Hub's one, and is the expected pattern on a wide screen.

5.4 Tablet

Same rail-plus-workspace model as landscape, permanently — not collapsible back into a Command Hub at any tablet size. Dashboard's stat cards move to a multi-column grid; create/edit sheets render as a centered fixed-width panel (~480px) instead of full width, so a form never stretches into a wide, half-empty sheet.

Breakpoints: Material 3's own 600dp/840dp window-size classes — not bespoke numbers. Full detail: Chapter 8.

5.5 Desktop

Status: 📄 **Draft** — the least-defined tier, sketched here as vision, not yet scoped for implementation (see Chapter 8's honesty note on desktop readiness before treating this as buildable).

Behavior: a professional, always-visible workspace layout, not a stretched tablet screen:

- **Sidebar** — the same rail concept as landscape/tablet, but at full desktop treatment: persistently expanded (icons + labels, never icon-only), positioned left.
- **Toolbar** — a persistent top strip above the workspace, holding contextual actions for whatever the workspace is currently showing (e.g. "New Warehouse," "Export," filters) — the desktop equivalent of a screen's `WFTopBar` actions, but positioned as a strip a mouse can reach quickly rather than a phone-style app bar.
- **Search** — a persistent, always-reachable way to jump straight to a Warehouse/Rack/Slot/Board/Offcut/Decor by name or code, without drilling down the hierarchy manually. **This is new scope** — no global search exists anywhere in the app today (the only search today is Decor-only, inside the Shopping List's threshold picker, Chapter 12). Treat this as a real, separate feature to design, not an assumed-trivial UI addition.
- **Workspace** — the main content area, to the right of the sidebar and below the toolbar; renders whichever module/screen is active, using the same adaptive multi-column patterns as tablet (§5.4) where the module supports it.
- **Status bar** — a persistent bottom strip for passive, glanceable state (e.g. sync/connection status, app version) — a desktop-only concept with no phone/tablet equivalent today. **Content is not yet defined** — flagged as an open question (§5.8), not assumed.

5.6 QR Scanner — exempt

Stays outside the Hub (and outside the desktop sidebar) entirely — it's plausibly the single most frequent action on an actual warehouse floor; burying it would cost a tap on the highest-frequency interaction in the app. Placement on phone (second FAB vs. leading `AppBar` icon) is open (§5.8); on tablet/desktop it's a persistent toolbar/sidebar action, always one click away, never nested in a menu.

Dashboard as home: the brand mark's "tap always returns to Dashboard" rule (Ch. 6.5), combined with Dashboard leading the module grid, makes **Dashboard the app's actual home screen**, replacing `WarehouseListScreen` (today's literal `main.dart` `home:`). A real, structural change — named explicitly here, not a side effect discovered mid-build.

5.7 Why not other navigation patterns

Documented so this doesn't get silently re-litigated later: a radial menu was considered and rejected (no established accessibility convention, worse one-handed reachability than a vertical grid, not a pattern any of this project's own reference brands use); a persistent bottom dock/tab bar was rejected (WoodFlow's structure is a drill-down hierarchy, not a set of 5–6 equal-weight peer destinations); hiding Scan inside any menu, radial or otherwise, was rejected for the frequency reasons in §5.6. Full record: Chapter 25.

5.8 Open questions

1. Hub button placement — bottom-center vs. bottom-trailing.
2. QR Scanner placement on phone — second FAB vs. leading `AppBar` icon.

3. ~300ms Hub-open animation — confirm the budget exception.
 4. Settings screen doesn't exist yet — needs its own scope before it can be built, separate from its Hub entry and motif.
 5. Role-based navigation — no Auth/Role system exists yet (v2.5). The module grid should be a plain descriptor list (id/icon/label/ route) now, so a future `modules.where((m) => role.canSee(m))` filter slots in later without restructuring the Hub — a seam, not a built permission system.
 6. Desktop search (§5.5) — scope, data source, and UI are all undefined; needs its own Phase 1 pass, not an assumption folded into general desktop work.
 7. Desktop status bar (§5.5) — content is undefined.
-

6. Visual Identity

Status: ● Direction approved, implementation not yet approved.

6.1 Illustration style

Monochrome, vector, blueprint/CAD/engineering-sketch line art — not photos, not colorful illustration, not cartoon graphics. Opacity 2–5%, never reducing readability. Built as procedural `CustomPainter` line art (lines/rects/arcs) — no illustration-asset pipeline exists in this project, and blueprints are themselves schematic simplifications, so this is a natural fit, not a compromise.

6.2 Module Illustration System — feasibility-graded

Every module gets its own subtle background illustration — this is the "illustration language" in one sentence: **each module is instantly identifiable by its motif alone, without reading a single word, while the motif itself stays too quiet to ever compete with real content.** Every illustration in this system, without exception, is: monochrome, blueprint/CAD-inspired, industrial, extremely subtle (2–5% opacity, §6.1), decorative only, and never interferes with usability or readability.

Module	Description	Motif	Feasibility
Warehouse	Warehouse racks, seen as a floor plan	Floor-plan aisle grid; isometric grid for "perspective" (not true 3D)	High
Rack	Shelving structure	Shelf-elevation lines + uprights	High
Slot	One storage cell, zoomed in	Bounded rectangle with corner tick marks	High
Board	Board storage — stacked panels	Stacked offset rectangles + dimension arrows	High — priority motif
Offcut	Offcut stack — irregular leftover pieces	Same rectangles as Board, one corner cut at an angle	High
Shopping List	Warehouse trolley	Flat cart schematic (deck + wheels + handle)	Medium
Calculators (Area)	Measuring tools — a dimensioned panel	Rectangle + dimension arrows	High
Calculators (Edge banding)	Measuring tools — a roll cross-section	Concentric arcs + tangent line	High
Dashboard	Warehouse overview	Bar-chart silhouette	High
Export	Technical document	Document silhouette, folded corner	High
QR Scanner	Scanner viewfinder	Viewfinder corner brackets + grid "label"	High — easiest and most recognizable

Module	Description	Motif	Feasibility
Settings	Warehouse blueprint	Blueprint grid + gear outline	High (once screen is scoped)
Forklift (if used, e.g. within Warehouse/Dashboard)	—	Flat side-elevation schematic, not a render	Medium, named simplification
Operator figure (if used, e.g. within Scanner)	—	ISO/safety-pictogram figure, not a cartoon	Medium, named simplification

6.3 Module accent colors

Decorative only — tints motif art + a thin header accent. **Never** used for buttons/status/anything a color-blind user must read unambiguously (those stay on Ch. 4.3's semantic roles). If a module accent and a semantic color ever compete, the semantic color wins.

Module	Accent (indicative)
Warehouse	<code>#1F4D3D</code> deep green
Rack	<code>#5B7A8C</code> steel blue-gray
Slot	<code>#6B7280</code> slate gray
Board	<code>#8A7561</code> warm taupe
Offcut	<code>#C08552</code> muted amber-orange
Dashboard	<code>#2C3E5C</code> navy
Shopping List	<code>#B8923A</code> muted amber
QR Scanner	<code>#3D7EA6</code> cyan-blue
Calculators	<code>#6B5B8C</code> muted violet
Export	<code>#78716C</code> neutral gray

6.4 Performance guardrails

One static `CustomPainter` per motif (`shouldRepaint => false`), `RepaintBoundary`-wrapped, never behind scrolling content — only fixed header/empty-state areas. Zero network assets, zero image decoding.

6.5 Brand mark

- **Buildable:** a typographic wordmark ("WoodFlow," module-accent green, deliberate tracking) + a simple geometric monogram (`CustomPainter` path, same blueprint register).
- **Not buildable well here:** a fully designed illustrative logo/icon asset — real graphic-design work outside procedural code; needs a human designer if that's the actual requirement.
- Placement: top-left, always visible. Phone: mark only. Tablet/desktop: mark + "WOODFLOW" wordmark. Tap always returns to Dashboard (Ch. 5.6).

6.6 Design Philosophy

Chapter 4 defines the literal token *values* (spacing numbers, radius numbers, elevation numbers). This section is the *why* behind those values — the philosophy a future decision should be checked against when a new situation isn't already covered by an existing token.

- **Color philosophy.** Color is used to mean something, never to decorate. A color always maps to either a semantic role (success/ warning/error/neutral, Ch. 4.3) or a module identity (Ch. 6.3) — never both at once, and never "just because it looks good here." The Forest Green seed exists because it's the one color already doing real brand work; everything else in the palette is low-chroma and quiet specifically so that seed keeps standing out when it matters.
 - **Typography hierarchy.** Hierarchy comes from the type scale (Ch. 4.2), never from inventing a one-off size or weight. If something needs to stand out, the question is "which existing role fits" before "what new style would look right" — a hierarchy with five deliberate levels reads faster than one with fifteen accidental ones.
 - **Spacing system.** One 4dp-based scale (Ch. 4.1), applied everywhere. The test for "does this need a new spacing value" is almost always "no" — six tokens cover a warehouse-management app's actual layout needs; a seventh value is a sign something else is wrong, not a gap in the scale.
 - **Border radius system.** Three values (Ch. 4.4), each tied to a role (chip/card/sheet), not to "what feels right" per screen. A consistent radius language is one of the cheapest, most effective ways an interface reads as "one product" instead of "assembled screens" — disproportionately valuable for how little it costs.
 - **Shadows.** Soft and low-opacity only — Material 3's own default elevation shadows, never a custom, heavier, or glossier shadow. Nothing in WoodFlow should look skeuomorphic or "lit from above with drama" — a shadow's only job here is to indicate stacking order, the same restrained role elevation itself plays (below).
 - **Elevation.** Elevation communicates stacking order (what's above what), never importance or decoration. Material 3 defaults only (Ch. 4.4) — an element doesn't get "promoted" with extra elevation to make it feel more important; that's what typography and layout position are for.
 - **Icon rules.** One concept, one icon, everywhere it appears (Ch. 4.4) — an icon is a label, not an illustration, and is judged on how fast it's recognized, not on how interesting it looks. The module motifs (§6.2) are where WoodFlow gets to be visually distinctive; icons are deliberately the boring, reliable, fast-to-read layer underneath that.
 - **Motion philosophy.** Motion explains a state change, it doesn't perform for the user. Every animation in Chapter 17 exists to answer "where did this come from" or "what just happened" — never "wouldn't this be a nice touch." The Command Hub's one deliberately longer animation (Ch. 5.2) is the single sanctioned exception, and it's sanctioned specifically because it's a rare, brand-defining moment, not a routine, repeated one.
-

7. Themes — Light & Dark

Status: 🟡 **Required, not yet implemented** — resolves a question left open in earlier drafts.

Today `main.dart` defines exactly one `ThemeData`; no dark theme exists.

- Add `darkTheme:` via the same Material 3 `colorScheme` mechanism (a second seed/brightness, not a second design system). Every `WF*` component already reads from `Theme.of(context)`, so this is "define the second `ColorScheme`," not "touch every screen."
 - Dark palette direction: low-chroma near-black **graphite**, not pure black (pure black + thin white lines reads harsh; graphite matches "professional machine" surfaces) with Forest Green carried through at adjusted lightness.
 - Module accents (Ch. 6.3) and motif opacity (Ch. 6.1) each need a second, separately-tuned value for dark.
 - Persistence: mirror `LocaleProvider`'s existing pattern exactly (a `ThemeProvider`, same `ChangeNotifier` /local-storage/ `ListenableBuilder` shape) — not a new state-management approach.
 - Theme choice is remembered permanently across launches.
-

8. Responsive Design — Phone / Tablet / Desktop

Status:  /  **Portrait/landscape/tablet direction approved via Chapter 5; Desktop is  draft.**

Configuration	Behavior
Phone portrait	Command Hub overlay (Ch. 5.2)
Phone landscape	Permanent nav rail + workspace (Ch. 5.3)
Tablet portrait/landscape	Permanent nav rail + adaptive content (Ch. 5.4)
Desktop	Sidebar + toolbar + search + workspace + status bar (Ch. 5.5) — least defined tier, see below

Breakpoints: Material 3's standard 600dp (compact/medium) and 840dp (medium/expanded) window-size classes — not a bespoke set, so the app rides a convention Flutter's own tooling already understands.

Desktop, grounded in fact: platform folders (`windows/`, `macos/`, `linux/`, `web/`) exist from initial project scaffolding, but **no desktop build has ever been exercised or tested** in this project's history — every verification has been Android. Not zero work, but not "just another breakpoint" either (does `sqlite` need the FFI variant there, does the camera-based QR scanner degrade sensibly with no camera). Sequenced **last** in any implementation phasing (Ch. 24), after mobile/tablet is proven.

Every layout should feel intentionally designed for its size, never merely stretched — the Dashboard's multi-column tablet layout and fixed-width tablet forms (Ch. 5.4) are the concrete examples of that principle in practice.

9. Warehouse Module

Status:  **Approved & Implemented** (Kroki 1–4).

The physical-location hierarchy: **Organization** → **Warehouse** → **Rack** → **Slot**.

- **Organization** — root of the hierarchy, minimal today (one default row, `defaultOrganizationId`), dormant until multi-tenant features (v2.5) need it. No auth, no billing, no multi-org UI.
- **Warehouse** — `name`, `address`, own QR code (`WF-W-#####`). Full CRUD, plus a hardened cascade-delete (`DeleteWarehouseUseCase`) that archives contained Boards/Offcuts and deletes Racks/Slots atomically, blocking partial deletion.
- **Rack** — belongs to a Warehouse, own QR code (`WF-R-#####`). No ledger history of its own (Racks/Slots are physically deletable, unlike Board/Offcut). `DeleteRackUseCase` gives it the same cascade-archive safety as Warehouse.
- **Slot** — belongs to a Rack, `capacity` (default 20, drives the fill-ratio grid), own QR code (`WF-S-#####`). `DeleteSlotUseCase` archives contained Boards/Offcuts before deleting the Slot.

All three deletion paths share their archive-row logic via `lib/data/usecases/cascade_archive_helpers.dart` rather than duplicating it three times — the concrete example of Chapter 20.5's "multi-entity atomic operations get dedicated use cases" rule.

10. Inventory Management

Status:  **Approved & Implemented** (Kroki 5–6, 8).

The actual stock entities: **Board** and **Offcut** — both self-sufficient (Offcut is not "a small Board" that needs a join to be usable), both reference the global **Decor** catalog by `decorId` (never a free-text code).

- **Board** — full, uncut panel. `slotId` is the *only* location field (Warehouse/Rack are always derived by joining `slot → rack → warehouse`, never stored redundantly). Status: `inStock` / `archived`. Lifecycle: create, move (`moveBoard`), location-only, no status change, archive (never physically deleted).
- **Offcut** — result of cutting a Board. `parentBoardId` for provenance; `decorId` is a denormalized copy of the parent Board's decor *at cut time*, so every Offcut screen shows a decor without joining to Board on every row. Status: `available` / `archived`. Lifecycle: `cutFromBoard` (does not resize or touch the parent Board — that's the future Cut Optimizer's job), move (`moveOffcut`), archive.
- **Ledger** (Krok 8) — append-only audit trail. Every create/move/ archive/cut/QR-regeneration event writes one `ledger_entries` row. **Never edited or deleted.** A correction is always a new entry referencing the previous one, never a rewrite — a standing, explicitly reconfirmed rule (Ch. 20.6).
- **Both** carry a QR code (Ch. 11) and full lifecycle history (`getLedgerForBoard` / `getLedgerForOffcut`), viewable on their respective detail screens.

Structurally locked, on purpose: Board/Offcut dimensions and `decorId` are not editable post-creation (Ch. 16.1 explains why).

11. QR System

Status:  **Approved & Implemented** (Krok 7).

- **Format:** `WF-{TYPE}-{code}` — `W`/`R`/`S`/`B`/`O` per entity type.
 - **Generation:** the first code is a **pure function of the entity's `id`** (first 8 hex chars, uppercase) — one source of randomness, not two independently-random values.
 - **Scanning** (`ScanScreen`) resolves any of the five entity types via `QrResolver` and navigates straight to the matching screen — the only place in the app that turns a camera frame into a navigation decision.
 - **Matching is case-insensitive**, normalized once at the resolver layer.
 - **Regeneration** (`regenerateQrCode()`) exists on all five entities but is explicitly flagged "admin-only" in comments only — **no enforcement, no UI exists for it today**. Board/Offcut regeneration writes a ledger entry (`qrRegenerated`); Warehouse/Rack/Slot simply overwrite the field (no ledger of their own, Ch. 9).
 - **Label printing** (Krok 7.3, `PdfLabelGenerator`) — PDF labels for any Board/Offcut/Slot selection, using a real embedded font (Noto Sans) so all 21 languages' diacritics render correctly.
-

12. Shopping List

Status:  **Approved & Implemented** (Krok 13).

- `Decor.minimumStockQuantity` (nullable `int`) — a piece-count threshold, `null` = never alert. Global per decor, not per warehouse (a documented future extension seam, not a limitation overlooked).
 - `ShoppingListService` aggregates current stock (Board+Offcut, excluding archived) per decor, compares to threshold, returns a sorted (most urgent first) list.
 - `ShoppingListScreen` is also the **only** place in the app a threshold is set or cleared — no separate decor-management screen exists. Its FAB opens a decor search, then a threshold sheet.
-

13. Calculators

Status:  **Approved & Implemented** (Krok 11).

Two pure, stateless calculators behind one tabbed screen — no `get_it` /interface, since there's exactly one reasonable implementation of each:

- **Area/Volume** (`BoardMeasurementCalculator`) — m^2/m^3 from length/width/thickness.
 - **Edge banding** (`EdgeBandingCalculator`) — roll length in meters from outer/core diameter and tape thickness (annular cross-section math). Calculation only — no roll-as-inventory tracking yet (a deliberate scope boundary; the math is a standalone function specifically so a future `EdgeBandingRoll` entity could reuse it).
 - Reached standalone (`WarehouseListScreen` /Command Hub icon) or as a shortcut from `BoardDetailScreen` / `OffcutDetailScreen` , pre-filled with that item's dimensions.
-

14. Export

Status:  **Approved & Implemented** (Krok 10).

- `ExportGenerator` interface, three `get_it`-registered implementations (`export_pdf` / `export_csv` / `export_rtf`) — `presentation/` never imports a concrete generator directly.
 - `ExportScreen` — pick a warehouse + format, export full inventory (Boards+Offcuts, with location/decor/status) via the system share sheet (`share_plus`, in-memory `XFile`, no disk write, works on web).
 - PDF export uses the same embedded Noto Sans font as label printing (Ch. 11) for full 21-language glyph support.
-

15. Dashboard

Status:  **Approved & Implemented** (Krok 9).

- `DashboardService` does all aggregation — the screen itself does **zero** computation, purely renders a `DashboardSnapshot`. Any future change adding a loop/sum/percentage directly in the screen is a regression of this boundary.
 - Shows: total boards/offcuts, overall fill rate, total racks/slots, and a stale-materials list (>1 year unmoved — the same threshold reused by Krok 14's AI query).
 - Per Chapter 5, Dashboard becomes the app's home screen once the Command Hub ships.
-

16. Entity Editing

Status: current-state facts (§16.1) are  verified. The forward-looking requirement (§16.2) is  Future Phase — Not Approved.

16.1 What's actually editable today

Entity	Repository-level (<code>copyWith</code>)	Actual UI today
Warehouse	<code>name</code> , <code>address</code>	No edit UI (delete UI only)
Rack	<code>name</code>	No edit UI; delete UI now exists (Ch. 9)
Slot	<code>name</code> , <code>capacity</code>	No edit UI; delete UI exists, now cascade-safe
Board	<code>slotId</code> (move), <code>status</code> (archive)	Move + Archive only
Offcut	<code>slotId</code> (move), <code>status</code> (archive)	Archive only — <code>moveOffcut()</code> has zero call sites in <code>lib/presentation/</code>
Decor	<code>name</code> , <code>minimumStockQuantity</code>	Only <code>minimumStockQuantity</code> , via Ch. 12's threshold sheet

Structurally locked, on purpose: Board/Offcut `decorId` /dimensions and Decor `code` / `manufacturer` are not `copyWith` parameters at all. An Offcut can never be verified larger than its parent Board if the Board's own dimensions could silently change after cutting; Offcut's `decorId` is a deliberate one-time copy specifically so it never has to stay in sync with a later-corrected Board. Decor's `code` is the material's actual manufacturer identity — a wrong catalog entry is a data-import problem (Ch. 20.7), not an in-app edit.

16.2 The forward-looking requirement

"Editing is a core feature — industrial environments constantly change, deleting and recreating entities should not be necessary." Agreed as a destination, covering Warehouse/Rack/Slot/Board/Offcut/ Shopping-List-entries/Minimum-Stock-entries (the last two both resolve to Decor, §16.1, not new entities) **plus "Export presets," which does not exist as a persisted entity today** — same treatment as Calculator presets (Ch. 24), a future candidate, not something to design rules for yet.

This requirement alone does not approve building editing UI — per Chapter 23, it needs its own Phase 1 spec, grounded the way §16.1 was, before any Phase 2 work.

17. Animations & Motion

Status: ● **Direction approved, not yet implemented.** Every entry below exists to answer "where did this come from" or "what just happened" — see Chapter 6.6's Motion philosophy for the underlying rule this catalog is checked against.

17.1 Screen transitions

Material's own default page-transition animation — already correct, no change proposed. The one addition: entity drill-down (Warehouse→Rack→Slot→Board/Offcut) uses a `Hero` shared-element transition on the entity's leading icon, so the icon visibly "travels" from the list row into the detail screen instead of the whole screen just being replaced — cheap, and it illustrates the app's real hierarchy rather than decorating an otherwise-unrelated transition.

17.2 Command Hub — opening and closing

The one deliberately longer, deliberately asymmetric-feeling-but- actually-symmetric motion in the app (Ch. 5.2):

- **Opening:** scrim fades in over the current screen → the WoodFlow button scales/morphs into the Hub panel, anchored at the button's own position → the module grid fades/slides in with a short (~20–30ms per tile) stagger.
- **Closing:** the exact reverse — module grid fades out first, the panel collapses back into the button shape, the scrim clears. It is not a generic "fade the dialog away" — it has to visibly collapse back into the same button it came from, so opening and closing read as one coherent object, not two unrelated animations.
- Duration: ~280–320ms total, the one deliberate exception to the ≤250ms budget (§17.8) — a rare brand moment, not a routine, many-times-a-shift action.

17.3 Dialogs & bottom sheets

A slightly more deliberate entrance curve than a plain instant appearance (`Curves.easeOutCubic` , ~200ms), applied once inside the shared `showWFConfirmationDialog` / `showWFBottomSheet` / `showWFDialog` helpers — every dialog and sheet in the app upgrades together, since they all route through the same few call sites. Closing mirrors opening, same curve in reverse, same short duration — no dialog gets a slower, more elaborate exit than its own entrance.

17.4 Cards

No motion on appearance in a scrolling list, deliberately. A stagger-in or fade-in per card feels premium on a short, one-time list — and works directly against fast scanning on a long, frequently-reloaded one (Shopping List, Slot grid). Cards appear instantly, fully rendered, the moment their data is ready. The only card-level motion in the system is a card's own use of the dialog/ sheet entrance (§17.3) *when it opens a form* — the card itself never animates in.

17.5 Loading

The existing centered, indeterminate `CircularProgressIndicator` (Ch. 4.5) — its own built-in rotation is the only loading motion in the system. No custom loading animation (skeleton screens, shimmer, pulsing

placeholders) is proposed — per Chapter 6.6's "explain a state change, don't perform," an indeterminate spinner already correctly communicates "working, no promise of when" without needing embellishment.

17.6 QR scan & search

- **QR scan:** a subtle scan-line sweep over the camera preview while awaiting a detection — real feedback that the app is actively looking, not decoration.
- **Search:** a ~150ms crossfade when the result count changes — small and cheap enough to never feel like it's blocking typing.

17.7 Explicitly rejected

Rejected	Why
Stagger-in on data lists	Fights fast scanning on a long list (Ch. 4.6) — see §17.4
Custom parallax / 3D route transforms	Highest jank risk on real devices; also exactly the "flashy" category the product vision explicitly avoids
Skeleton/shimmer loading placeholders	Not proposed — see §17.5; an indeterminate spinner already says everything that needs saying

17.8 Hard budget

Every animation $\leq 250\text{ms}$ except the Command Hub's ~300ms brand moment (§17.2); every animation respects `MediaQuery.disableAnimations` (system reduce-motion) by snapping straight to the end state.

18. Accessibility

Status:  **Approved principles**, applying to all future work regardless of a given feature's own status.

- Color is never the only signal — the fill-ratio chip's numeric label alongside its color is the reference example.
 - Every animation respects reduce-motion (Ch. 17).
 - Status/semantic color (`WFStatusChip`) stays independent of module accent color (Ch. 6.3) — a color-blind user's "something needs attention" reading must never be entangled with which module they're in.
 - Touch targets: Material's 48×48dp default, never shrunk for density.
 - Text scaling: theme roles (Ch. 4.2) respect the system font-scale setting automatically.
 - Icon-only controls (Hub button, phone-size logo mark) get explicit `Semantics` /tooltip labels.
-

19. Performance Rules

Status:  **Approved principles.**

- Motif rendering (Ch. 6): one static `CustomPainter` per module, `shouldRepaint => false`, `RepaintBoundary`-wrapped, never behind scrolling content.
 - Command Hub: one coordinated `AnimationController`, GPU-friendly properties only (opacity, scale, position) — no per-frame layout recalculation.
 - Micro-interaction budget: $\leq 250\text{ms}$ everywhere except the Hub's one brand moment (Ch. 17).
 - Explicitly rejected for cost: stagger-in list animations, custom parallax/3D route transforms.
 - 60 FPS target: nothing proposed anywhere in this handbook does per-frame Dart-side work during a gesture — the cost profile is "draw once, animate GPU-composited properties."
 - No desktop-specific performance work proposed yet (Ch. 8) — that gets its own pass once desktop is an actual phase.
-

20. Flutter Architecture

Status:  **Approved & Implemented.**

20.1 Layering

`domain/` (entities, repository interfaces, services, use cases) never imports `data/` or Flutter. `data/` implements repositories/use cases against `DatabaseService`. `presentation/` resolves dependencies via `sl<T>()` and only ever imports the `domain/` interface, never a concrete `data/` class.

20.2 `get_it` as the sole DI mechanism

No class has its own `static instance` / private singleton constructor — every class takes plain constructor dependencies; `get_it` (`service_locator.dart`) owns instance lifecycle. Riverpod was considered and deferred (more idiomatic, but a full state-management rewrite for a marginal gain at this project's size). Manual constructor injection without a container was rejected — unreadable past 5+ dependencies at this project's module count.

20.3 `Result<T>` over exceptions at layer boundaries

Exceptions are thrown only inside `data/`; every repository method catches them once and converts via `mapExceptionToFailure()`; every repository method returns `Result<T>`, never a bare `T` that might throw; `presentation/` calls `result.when(success:, failure:)`, never a manual `try/catch`. `Either<Failure, T>` (`dartz`) was considered and rejected — an external dependency for no real gain over a project-owned type.

20.4 Migrations as versioned files

Each schema version is its own file implementing `Migration` (`version`, `up(Database db)`); `DatabaseService` contains zero SQL, delegating entirely to `MigrationRunner`. Migrations are append-only — a shipped file is never edited; fixes ship as a new migration. No rollback (`down()`) yet — no production user data exists to roll back.

20.5 Multi-entity atomic operations get dedicated use cases

`sqlite` doesn't support nested transactions on one connection, so an operation touching several tables atomically can't call several repositories' own methods from inside each other. Instead: a dedicated class in `data/usecases/` operating directly on `DatabaseService.transaction()`. Current examples: `DeleteWarehouseUseCaseImpl`, `DeleteRackUseCaseImpl`, `DeleteSlotUseCaseImpl`, sharing archive-row logic via `cascade_archive_helpers.dart`.

20.6 Domain invariants

Enforced structurally unless noted:

- A Board/Offcut has exactly one location field — two locations at once is structurally impossible.
- Every ledger transaction has a timestamp and source (`NOT NULL` columns).
- **The ledger is immutable — INSERT-only, by convention.** No code ever `UPDATE`s or `DELETE`s a `ledger_entries` row; a correction is always a new entry referencing the previous one.
- A UUID never changes after creation — no `copyWith` exposes an `id`.

- Archive never deletes history.
- **Documented, not yet enforced:** an Offcut can't be larger than its parent Board (`cutFromBoard()` doesn't check yet); a Board with an active Offcut shouldn't itself be archivable (`archive()` doesn't check yet).

20.7 Accepted future architecture (not built)

- **Printer integration** — printing always optional, never a dependency; app stays fully usable with zero printer configured. `PrinterService` abstraction, same interface+ `get_it` shape as `LabelGenerator` / `ExportGenerator`. Full record: Chapter 25.
 - **Smart Offcut Scoring Engine** — v1 (today) collects data only; v2.x adds a deterministic, fully configurable rule-based score (Keep/Review/Scrap), always a recommendation, never automatic; v3.0 adds learning from recommendation-vs-outcome history. Full record: Chapter 25.
-

21. Coding Standards

Status:  **Approved & Implemented.**

21.1 Localization (21 languages)

`app_p1.arb` is the template; new keys are authored there first, then hand-authored into the other 20 — no translation tooling exists. `test/arb_consistency_test.dart` enforces exact key-set parity, `@@locale` matching, and placeholder-name parity across every file. Quality is honestly tiered (`docs/LANGUAGE_QUALITY.md`): strong (en/pl/de/fr/it/es/ru), needs-review (11 languages), requires-review-before-production-use (ga/cy/gd).

21.2 Testing

- Pure/stateless services — direct unit test, no DB.
- DB-backed services/repositories/use cases — real `*RepositoryImpl` against an in-memory `sqlite_common_ffi` database.
- **Widget tests touching a repository — in-memory Dart fakes, never the real DB-backed repository** — `sqlite_common_ffi` deadlocks against `flutter_test`'s fake-async pump loop (confirmed this project, `tester.runAsync()` did not fix it). The DB-backed test stays the source of truth for persistence correctness.

21.3 QR codes

See Chapter 11 for the full convention; the coding-standard summary is: one source of randomness (`id`-derived), one normalization point (`QrResolver`), never re-implemented per call site.

21.4 General conventions

- No Riverpod anywhere — plain `get_it` + `StatefulWidget`.
 - One file per language/manufacturer/format where that pattern applies (`data/ai_patterns/`, `data/decor_seeds/`, `data/export/*Generator`) — adding a new one is a new file + one registration line.
 - Sentinel objects (`_unset`) distinguish "leave field unchanged" from "explicitly clear to null" in every entity's `copyWith` with a nullable field.
-

22. Folder Structure

Status:  **Approved & Implemented** — reflects `lib/` as it exists today.

```
lib/
  core/
    constants/      - AppConstants (dbVersion, etc.)
    errors/         - exceptions.dart, failures.dart (Result<T>)
    events/         - domain_event.dart, event_publisher.dart
    services/       - service_locator.dart (get_it), locale_provider.dart
  data/
    ai_patterns/    - one file per language (Krok 14)
    database/
      migrations/   - one file per schema version (v1..v8)
    decor_seeds/    - one file per manufacturer (Krok 12)
    export/         - PdfExportGenerator, CsvExportGenerator, RtfExportGenerator
    labels/         - PdfLabelGenerator
    repositories/   - *RepositoryImpl (implements domain/repositories/*)
    usecases/       - multi-entity atomic operations (Delete*UseCaseImpl, cascade_archive_helpers.
  domain/
    entities/       - pure data classes, zero Flutter/data imports
    events/         - BoardArchived, OffcutArchived, etc.
    repositories/   - interfaces only
    services/       - AiQueryEngine, DashboardService, ShoppingListService, calculators, etc.
    usecases/       - Delete*UseCase interfaces
  l10n/            - app_*.arb (21 languages) + generated AppLocalizations
  presentation/
    <one folder per screen area>/ - warehouse/, rack/, board/, offcut/,
                                   dashboard/, shopping_list/, calculators/,
                                   export/, ai_query/, scan/, debug/
    design_system/  - WF* component library (Ch. 4)
```

No `location/` module exists (an empty leftover directory, not a real feature — flagged here only so it isn't mistaken for something undocumented).

23. Development Workflow

Status:  **Approved & Implemented process.** The durable governance this workflow implements — roles, review, and approval sequence, independent of any specific tool — is defined separately in `docs/project/Decision_Framework.md`, not restated here.

23.1 Specialist workflow

Before answering, determine which specialist(s) — from `.claude/skills/` (`woodflow-architect` , `flutter-expert` , `database-architect` , `warehouse-expert` , `wood-industry-expert` , `ui-ux-reviewer` , `design-system-expert` , `mobile-ux-expert` , `code-reviewer` , `performance-reviewer` , `security-reviewer` , `ai-architect` , `product-manager` , `website-expert`) — are relevant, and think from their perspective. When code is generated: review architecture, code quality, performance, and security before the final answer. **The Expert System is Claude Code development infrastructure only — never a runtime feature of the app.** Nothing under `.claude/skills/` is imported or run by `lib/` code (a corrected earlier mistake — full record: Chapter 25).

23.2 Phase 1, then Phase 2 — never skip Phase 1

For any new or significantly changed UI/visual feature:

Phase 1 (required, every time): written specification + reasoning

- UX explanation + performance impact + a concrete mockup for anything visual. Wait for explicit approval.

Phase 2: implementation, testing, refinement — only after approval.

"Approve the direction" is not "implement it." If a reason to deviate from an approved spec appears mid-build, stop and explain before making the change.

Before writing a Phase 1 spec as a *new* document or a *new* handbook chapter, check Chapter 2's single-source-of-truth rule first — extend existing content if this handbook already covers the topic, rather than starting a new one.

23.3 Verification before commit

Every implementation phase ends with `flutter analyze` (0 issues), `flutter test` (full suite green), `flutter build apk --debug` (succeeds) — reported as an explicit checklist. Commit only after explicit approval, even when verification is clean.

23.4 Commit discipline

New commits, not amended ones, except when explicitly requested. Focused, atomic commits — unrelated changes are separate commits, even in the same session.

24. Future Ideas

Status: 🚧 **Future Phase — Not Approved.** None of the below are scheduled. None may be implemented without a separate, explicit decision to pull a specific item into an actual roadmap step. Per Principle 9 (Ch. 3), nothing here affects current production.

24.1 From the product backlog (docs/BACKLOG.md)

This table is a condensed summary, kept here for at-a-glance scanning. docs/BACKLOG.md holds the original, fuller entries (problem/value/dependencies/acceptance criteria) and is the detailed source — if the two ever visibly diverge, the header's rule applies and this handbook wins, but the summary below should be corrected to match docs/BACKLOG.md 's intent rather than treated as a competing opinion about the idea itself.

Idea	Priority	One-line problem
Demo Mode	High	Fresh install is empty and unconvincing
WoodFlow Academy	High	No in-app guidance across 14+ feature areas
Installation Wizard	High	First launch has no language/units/QR/printer setup flow
Health Check	Medium	Data-quality issues accumulate silently
Diagnostics	Medium	No way to see DB/migration/sync/printer state without dev access
Works with WoodFlow	Medium	No certification path for hardware vendors
Feature Preview	Medium	Higher-tier features would dead-end FREE users
Confidence Level (future AI)	Low	A future scored recommendation needs a reliability indicator
Explain AI (future AI)	Low	A future recommendation needs visible reasoning

24.2 Accepted-direction, not-yet-built architecture

- **Printer integration and Smart Offcut Scoring Engine** — see Chapter 20.7 for the approach, Chapter 25 for the full decision record.

24.3 From this handbook's other chapters

- **Cut Optimizer** — the next actual roadmap step (Krok 15), not yet started.
- **Desktop platform adaptation** (Ch. 8) — sequenced last.
- **Settings screen** (Ch. 5.8) — no scope defined beyond its Hub entry and motif.
- **Onboarding** — no flow exists; its own candidate, not folded into visual-identity work.
- **Generic metadata / key-value entity fields** — rejected for v1 as a speculative abstraction; a real, named field gets its own typed column when actually needed.
- **Board/Offcut dimension or decor correction workflow** — locked fields today (Ch. 16.1); a real need would get its own explicitly-logged correction action, not a generic edit field.
- **Decor code/manufacturer correction workflow** — same reasoning.
- **Role-based navigation/permissions** — seam kept open (Ch. 5.8), no Auth/Role system to build against yet.

- **Brand language beyond the app** — reports, exported PDFs, QR labels, website, desktop application, marketing materials. Two (PdfLabelGenerator , ExportGenerator) already exist in this codebase and are legitimate near-term extensions once Chapter 6 is built; website/desktop/marketing are different codebases and teams entirely.
 - **A fully designed logo/icon brand asset** (Ch. 6.5) — real graphic-design work, not something procedural code produces well.
 - **Export presets** — do not exist as a persisted entity (Ch. 16.2).
-

25. Decision Log

Condensed record of binding architectural/product decisions. Full reasoning for the ADR-sourced rows lives in `docs/adr/` (not moved to archive — ADRs remain their own permanent record type).

Decision	Chosen	Rejected alternative(s)	Why
Dependency injection	<code>get_it</code> , sole mechanism (Ch. 20.2)	Riverpod; manual constructor injection	Riverpod = full rewrite for marginal gain now; manual DI unreadable past 5+ deps
Error handling	<code>Result<T></code> at every repository boundary (Ch. 20.3)	Exceptions to UI; <code>Either<Failure, T></code> (<code>dartz</code>)	<code>Result<T></code> makes fallibility visible in the signature without an external FP dependency
Schema evolution	One versioned file per migration, append-only (Ch. 20.4)	Single switch-statement method; external ORM migration tool	Append-only file history is reviewable; ORM adds a dependency not needed yet
Multi-entity atomic ops	Dedicated <code>data/usecases/</code> classes on raw <code>DatabaseService.transaction()</code> (Ch. 20.5)	Calling repositories from inside each other	<code>sqflite</code> has no nested transactions per connection
Ledger mutability	Append-only, INSERT-only, corrections are new entries (Ch. 20.6)	Editing/deleting history rows	The Ledger's entire value is being a trustworthy, tamper-proof audit trail
Warehouse/Rack/Slot delete	Cascade-archive use cases, atomic, all three sharing one helper (Ch. 9)	Bare <code>repository.delete()</code> reachable from UI	The original Rack/Slot delete was unconditional with no occupancy check — a real data-integrity gap, fixed
Board/Offcut identity fields	Structurally non-editable (<code>decorId</code> , dimensions) (Ch. 16.1)	Generic edit form covering every field	Protects the Offcut-can't-exceed-parent-

Decision	Chosen	Rejected alternative(s)	Why
			Board invariant and the decor-copy-at-cut-time guarantee
Krok 14 (AI) scope	Fully deterministic pattern-matching, zero LLM/network calls	Any predictive/learning logic	Matches the project's "no unverified predictive logic" rule, consistent with the Scoring Engine's own v1/v2/v3 staging
Expert System	Claude Code dev-process only (<code>.claude/skills/</code>), never runtime app code	Real Dart runtime classes (<code>lib/domain/experts/</code>)	Corrected mid-session — the first version was an architectural mistake, deleted before commit
Notes/metadata (entity editing)	Single <code>notes</code> field per entity if ever built; generic metadata rejected	A schema-less key/value bag	Matches "never create unnecessary abstractions" — a named need gets a named column
Printer integration	Optional layer, <code>PrinterService</code> abstraction, never a dependency	Required step in label lifecycle; single-brand-first with no interface	App must stay fully usable with zero printer configured; interface-first avoids a rewrite when brand #2 is added
Offcut scoring	Three-stage path (v1 data collection → v2 deterministic configurable rules → v3 learned adjustment), always operator-final-decision	Jumping straight to ML	No training data exists yet (v1 collects it); configurable rules stay explainable before any learning layer
Illustration style	Procedural blueprint/CAD <code>CustomPainter</code> line art (Ch. 6.1)	Hand-illustrated pictorial scenes	No illustration-asset pipeline exists; blueprints' own schematic simplicity

Decision	Chosen	Rejected alternative(s)	Why
			matches what code-drawn paths can do well
Navigation	WoodFlow Command Hub (overlay portrait, rail landscape/tablet) (Ch. 5)	Radial menu; persistent bottom dock; hiding Scan inside a menu	Radial/dock don't match a drill-down hierarchy or this app's own reference brands; Scan is too high-frequency to bury
Dark mode	Required (Ch. 7)	Light-only, deferred indefinitely	Explicit product requirement; WF* components already theme-driven, low marginal cost

26. Changelog

The living, dated changelog remains canonical at `docs/CHANGELOG.md`. The dated log entries themselves are **not** duplicated here — only the roadmap status table below, which mirrors `docs/CHANGELOG.md`'s own status header, is repeated for convenience. `docs/CHANGELOG.md` is authoritative for status; if this table and that file ever disagree, `docs/CHANGELOG.md` is wrong and needs updating to match reality, the same as this table would.

Stage 1 / FREE roadmap (15 steps):

Krok	Feature	Status
1	Organization	✓
2	Warehouse	✓
3	Rack	✓
4	Slot	✓
5	Board	✓
6	Offcut	✓
7	QR (generate, scan, PDF labels)	✓
8	History/Ledger	✓
9	Owner Dashboard	✓
10	Export PDF/CSV/RTF	✓
11	Calculators	✓
12	Decor catalog (EGGER, 421 entries)	✓
13	Shopping list / low stock	✓
14	AI v1 (deterministic NL query)	✓
15	Cut Optimizer	🟪 Not started

Since Krok 14 (this handbook's own session): Warehouse deletion cascade hardening; WoodFlow Design System (Ch. 4) built and applied to every screen; Rack/Slot delete cascade-archive fix (Ch. 9); Visual Identity, Command Hub, Entity Editing, and Product Vision drafted (Ch. 5, 6, 16); this handbook created, consolidating all of the above.

Archive

Source documents this handbook was assembled from are preserved at `docs/archive/` (design-system drafts) and `docs/adr/` (architecture decision records, kept in place as their own permanent record type — not moved). `docs/BACKLOG.md`, `docs/INVARIANTS.md`, `docs/LANGUAGE_QUALITY.md`, `docs/QR_CODES.md`, and `docs/CHANGELOG.md` also remain in place as their own living references, cross-linked from this handbook rather than duplicated into it.